



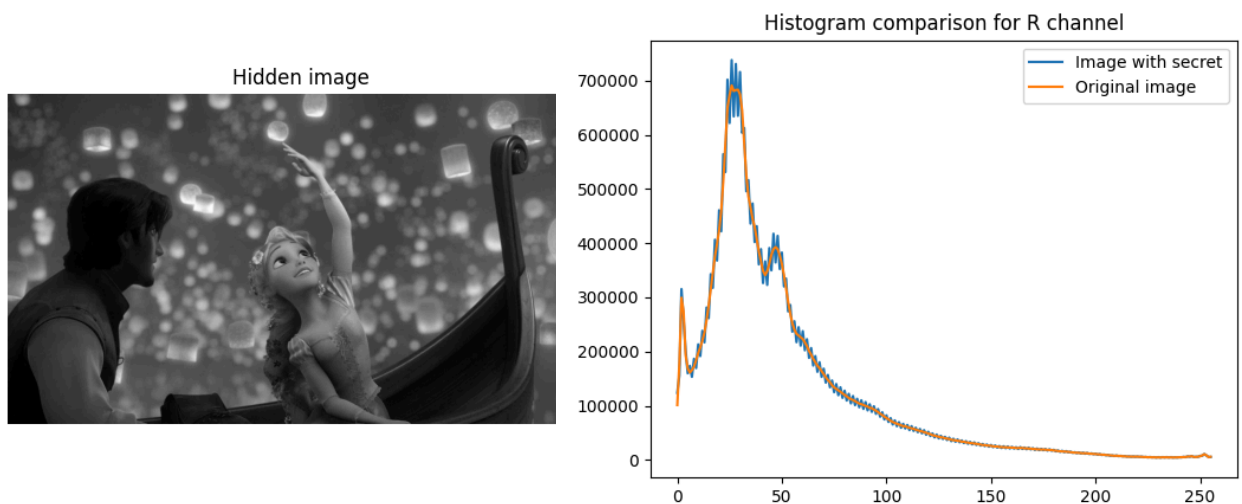
UNIVERSITY OF GENEVA

Faculty of Science

LSB steganography

Digital Forensics

14x065



Michel Jean Joseph Donnet

2025-11-14



Contents

1	Introduction	3
2	Methodology	4
2.1	Steganography vs. Cryptography	4
2.2	Steganography in images	4
3	Implementation	6
4	Results	7
4.1	Bit-plane image	7
4.2	Binary image hidden in RGB image	8
4.3	Binary image hidden in RGB image	9
4.4	Complete grayscale image hidden in RGB image	9
4.5	Robustness tests	11
4.5.1	RGB image	11
4.5.2	Grayscale image	12
5	Discussion	13
5.1	Binary image hidden	13
5.1.1	Metrics	13
5.1.2	Pixel-wise difference	14
5.1.3	Histograms	14
5.1.4	Robustness	15
5.1.4.1	RGB image	15
5.1.4.2	Grayscale image	15
6	Conclusion	16
	Bibliography	17



1 Introduction

Since the dawn of time, humans have tried to secretly transmit information. For instance, Caesar used encrypted messages to his subordinates in order to prevent his plans being discovered by enemies. Similarly, during the World War II, German transmitted coded messages to share instructions. However, their encoded communications were discovered, which led, among other things, to the loss of the war. Encryption is good, but it's visible and breakable: the others can see that there is an encrypted exchange of messages. That is why people began to hide information in various ways, such as in messages, images, audio, etc, what is known as steganography.

In this work, some basic steganography methods will be implemented and tested.



2 Methodology

First of all, what is steganography and what are its advantages and disadvantages compared to cryptography?

Definition 1. “Steganography is the art of hiding information in ways that prevent the detection of hidden messages”.^[1]

2.1 Steganography vs. Cryptography

Unlike the steganography, the cryptography is the art of securing a communication in the presence of hostile behavior.¹ So it implies that the adversarial part can see the communication, but not the content of the communication.

At the difference, according to the Definition 1, the steganography hide the communication in such a way the adversarial part don't see at all the communication. This way, information can be conveyed without drawing attention to it.

So the main advantage of the cryptography lies in the fact that the adversarial part is unable to decrypt the communication. However, if the adversarial part can decrypt the communication, this may result in loss of war, as shown by German during World War II with their encryption machine Enigma that was decrypted by British. And the main advantage of the steganography lies in the fact that the adversarial part don't see at all the communication. Nevertheless, if communication is discovered, this could also cause serious problems, like for cryptography.

2.2 Steganography in images

To properly see an image on a computer, each pixel intensity of the image is encoded. Then, the monitor activate each 'small led' according to the pixel intensity values in such a way the image is properly printed on the screen. Depending on the image representation, pixel intensity can be encoded with more or less bits. In our case, we will work with images encoded by only 8 bits, so with a range of possible intensity given by $[0, 255]$. Due to the imperfection of the human visual system that is less sensitive to small details, LSB of the image (Least Significant Bit) will be used to store the secret: the message will be coded in binary in the LSB of the image. If you take a look at the Figure 1, you can easily see that most of image information are stored into Most Significant Bit (MSB: level 7) of the image. On the contrary, only small details are stored in the LSB (level 0), that appears as some “noise”. In this way, the secret message will provoke only small intensity variations of pixels, which is undetectable for the human visual system. And it could be easily retrieved by the knowledge of the existence and position of the message in the image.

Color images use 3 channels: channel R that represents all red colors of the image, channel G that represents all green colors of the image and channel B that represents all blue colors of the image. Thanks to this channels and to the additive property of light colors, all colors can be represented. For instance, taking maximum intensity of both channels will be white color.

All the color image channels are only matrix of pixel intensity values. In this way, all channels can be used to hide some message into LSB. Depending on the size of the message and

¹ Wikipedia



on the channel chosen, human visual system could not see any difference between original image and image with hidden secret. For instance, human visual system is less sensitive to blue color than to green color because it contains more cones sensitive to green than cones sensitive to blue. (40% vs 5%) In this way, the blue channel can be used to encrypt some message to render the message more undetectable by human visual system.

So steganography can hide some message in an image in an undetectable way for human visual system.



3 Implementation

An implementation of steganography method on image is implemented. The documentation is given in Listing 1.

```
usage: tp5.py [-h] [-i IMAGE] [-b] [-m MESSAGE] [-mi MESSAGE_IMAGE] [-msb
MSB_NUMBER] [-g] [-k KEY]
```

Steganography

options:

```
-h, --help            show this help message and exit
-i, --image IMAGE     Path to base image
-b, --bit_plane       Display each bit-plane of the image
-m, --message MESSAGE
                        Message to hide
-mi, --message_image MESSAGE_IMAGE
                        Image to hide
-msb, --msb_number MSB_NUMBER
                        Store the x MSB of grayscale image into LSB of the RGB
```

base image

```
-g, --grayscale       Convert base image to grayscale
-k, --key KEY         Key used to hide message in pseudo-random positions
```

Listing 1: Documentation

In this way, if you want to hide a string message to the image, simply execute the code like bellow:

```
python tp5.py -i "..\images\bg_image.jpg" -m "This is a secret message !"
```

And if you want to hide a binary image into the base image, execute the code like bellow:

```
python tp5.py -i "..\images\bg_image.jpg" -mi "..\images\test_image.jpg"
```

And finally, if you want to hide an image by conserving the 8 Most Significant Bit (MSB) of the image to hide, execute the code bellow:

```
python tp5.py -i "..\images\bg_image.jpg" -mi "..\images\test_image.jpg" -msb 8
```

For the implementation, some pseudo-random number generator are used to hide message in pseudo-random positions in the image. This pseudo-random number generator take an initial number as initialization, and generate a sequence of pseudo-random numbers that will be always the same sequence if the same initial number is given. In this way, an adversarial part can detect that there is a message in the image, but can't obtain the message unless he finds the encryption key that is used to generate pseudo-random positions. And on top of that, as we work with pseudo-random numbers, we can encrypt and decrypt thanks to the same sequence generated by the same initial number.

In the implementation, positions are chosen pseudo-randomly according to the initialization of the random generator. A position can't be chosen twice, and the sequence of positions chosen is dependent of the given list of index: a smaller list of index will not make a choose of same index, even if the index chosen in the big list of index are present in the smaller list of index. In this way, passing a smaller list of index result in a desynchronization of index

pseudo-randomly chosen, which makes the message impossible to retrieve. So a proper work has been made on pseudo-random choice of index.

4 Results

4.1 Bit-plane image



Figure 1: Bit-plane decomposition of an image

4.2 Binary image hidden in RGB image

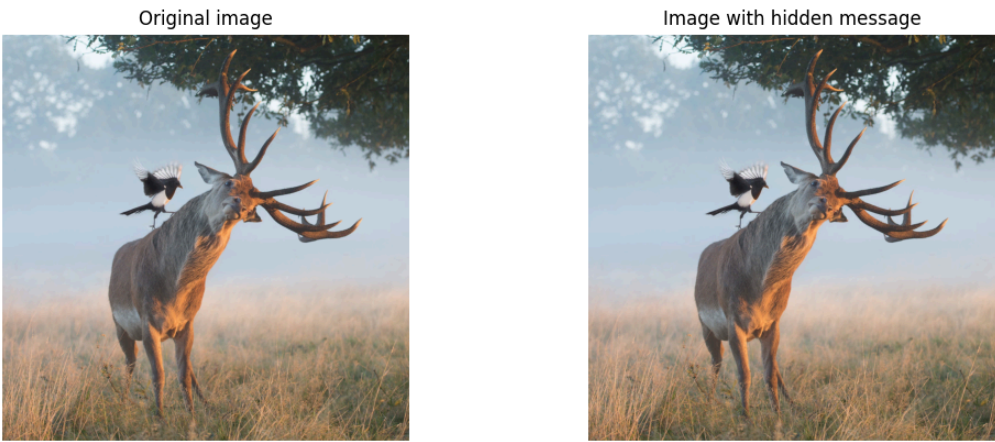


Figure 2: Binary image hidden

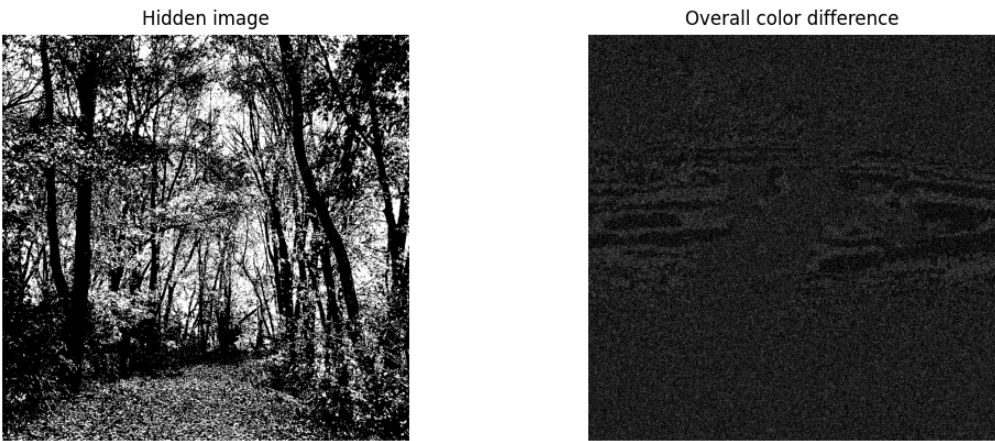


Figure 3: Binary image hidden

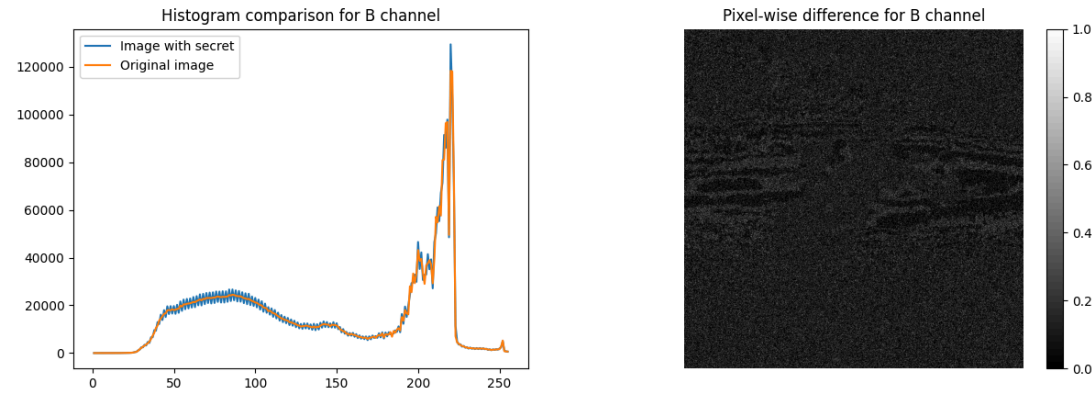


Figure 4: Impact of hidden message on RGB image



```

-----
Image comparison between original image and image with secret
-----
----- OVERALL COMPARISON -----
-----
MSE: 0.041503
PSNR: 61.950008705086894
SSIM: 0.9995570668923494
-----
----- CHANNEL R -----
-----
MSE: 0.0
PSNR: inf
SSIM: 1.0
-----
----- CHANNEL G -----
-----
MSE: 0.0
PSNR: inf
SSIM: 1.0
-----
----- CHANNEL B -----
-----
MSE: 0.124509
PSNR: 57.178796157890275
SSIM: 0.9986712006770483
-----

```

Listing 2: Output of the code

4.3 Binary image hidden in RGB image

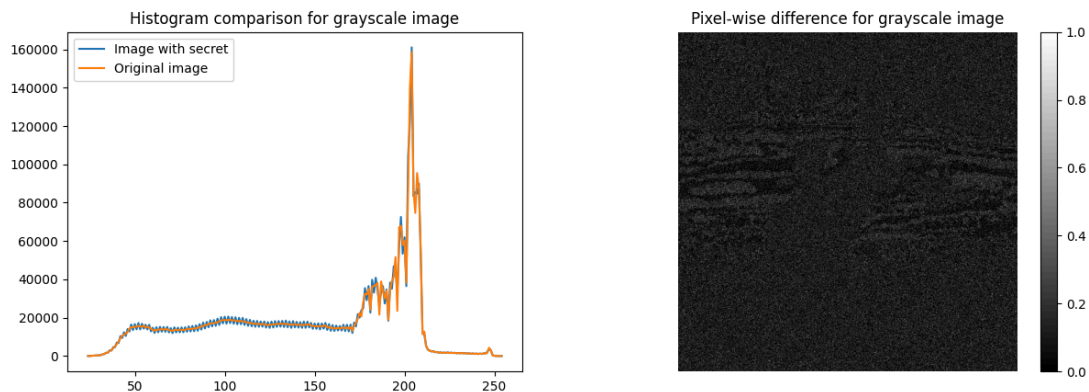


Figure 5: Impact of hidden message on grayscale image

```

-----
Image comparison between original image and image with secret
-----
-----
MSE: 0.12483775
PSNR: 57.16734428264796
SSIM: 0.9986605071997674
-----

```

Listing 3: Output of the code

4.4 Complete grayscale image hidden in RGB image

The method implemented consist to hide multiple bit-plane in LSB of each color channel of support image. It works fine if the base image is much bigger than the image to hide, because more than 1 bit-plane are hidden in LSB of the base image.

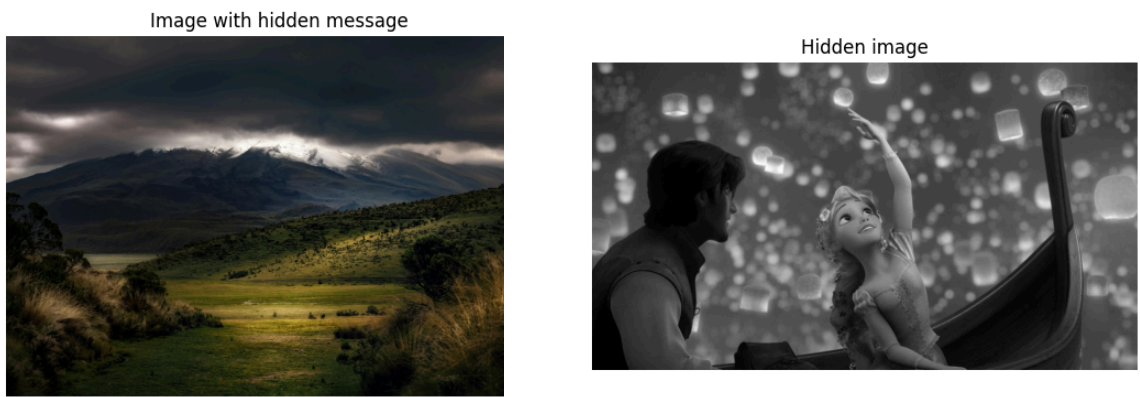


Figure 6: Grayscale image hidden

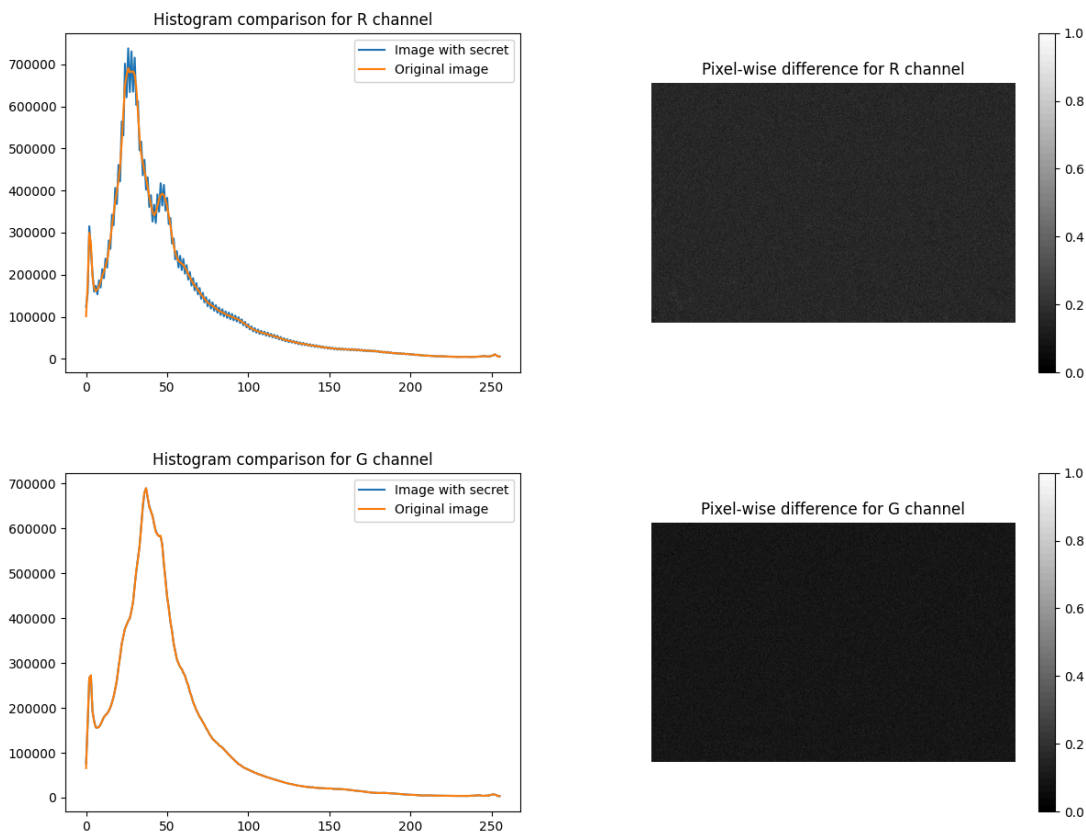


Figure 7: Grayscale image hidden

```

-----
Image comparison between original image and image with secret
-----
----- OVERALL COMPARISON -----
-----
MSE: 0.08855482506790993
PSNR: 58.658681313861024
SSIM: 0.9991432550816931
-----
----- CHANNEL R -----
-----
MSE: 0.1659685442900714
PSNR: 55.93054576055208
SSIM: 0.9984012127967222
-----
----- CHANNEL G -----
-----
MSE: 0.09969593091365841
PSNR: 58.14402927910395
SSIM: 0.9990285524483571
-----
----- CHANNEL B -----
-----
MSE: 0.0
PSNR: inf
SSIM: 1.0
-----

```

Listing 4: Output of the code

4.5 Robustness tests

4.5.1 RGB image

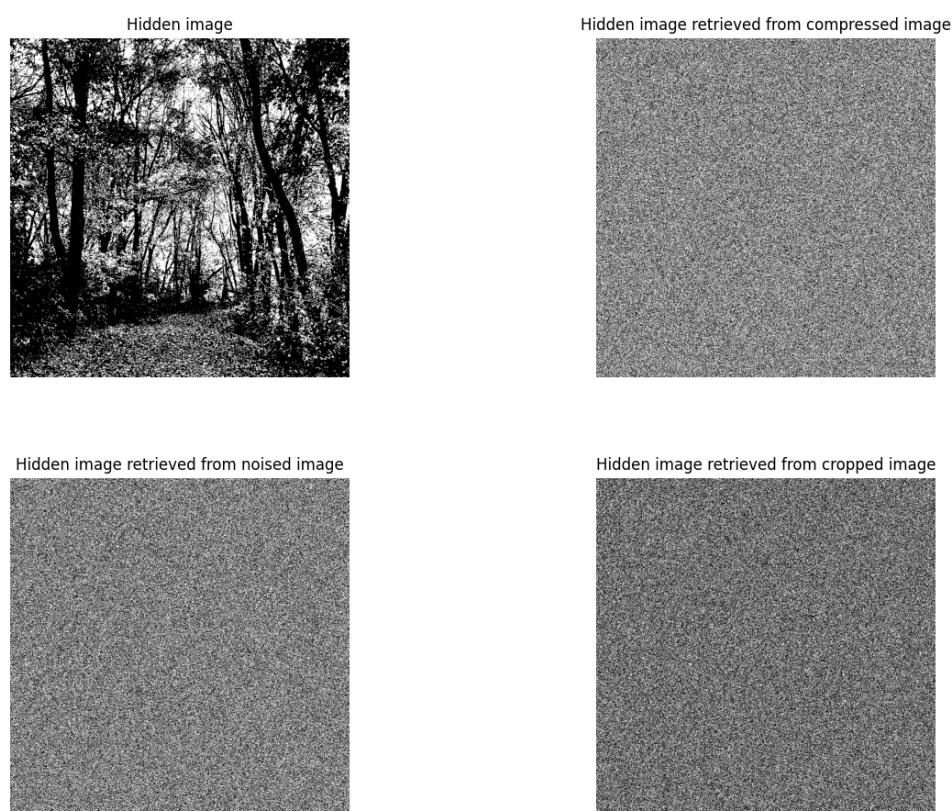


Figure 8: Robustness tests on RGB image

4.5.2 Grayscale image

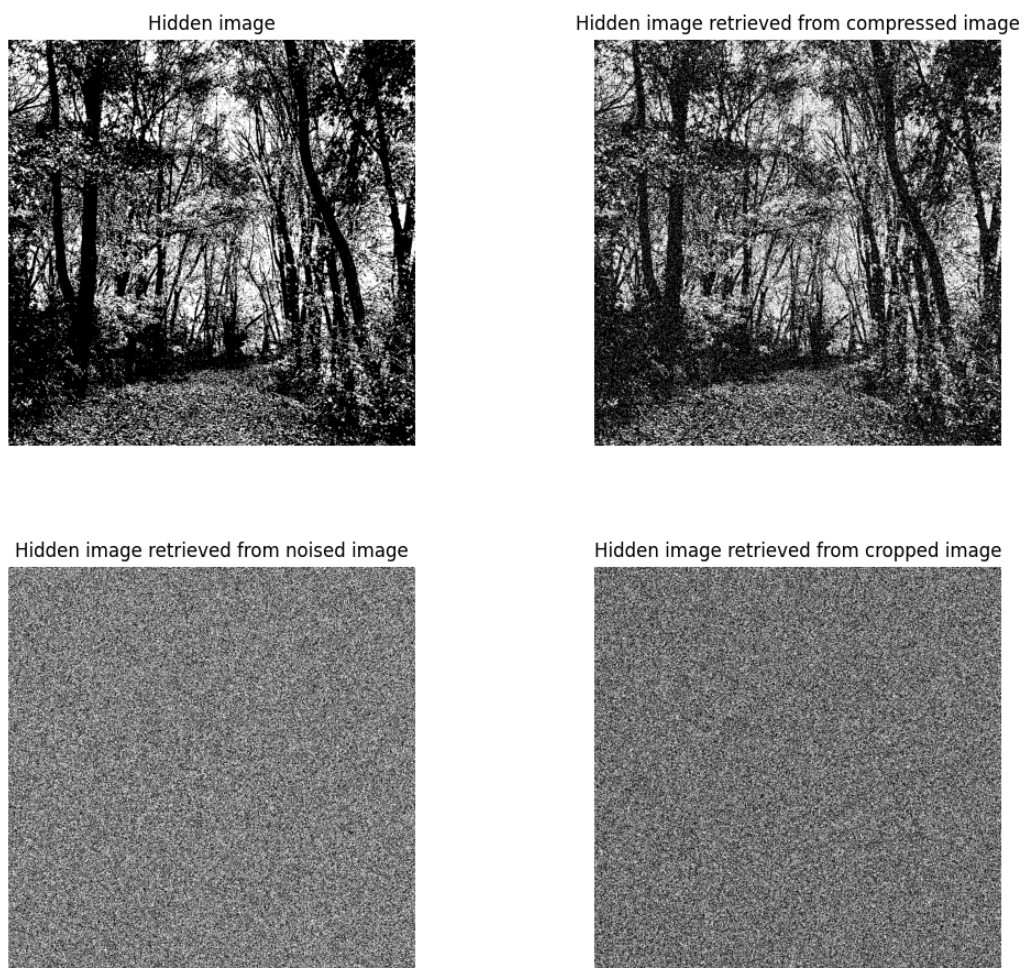


Figure 9: Robustness tests on grayscale image



5 Discussion

As explained in Section 2, each image can be decomposed into bit-planes. The Most Significant Bit of the image contains a lot of useful informations, because we can easily see on Figure 1 that for level 7 (corresponding to MSB), it's possible to see what is the picture. On the contrary, for level 0 that correspond to Least Significant Bit, only some noise appears except in smooth area. This is due to the fact that small details are encoded in LSB of the image. In this way, modifying the LSB of an image will have only a small impact on the visual quality of the image, especially since human visual system is less sensitive to small details.

5.1 Binary image hidden

2 cases have been tested here:

- a test with RGB base image
- a test with grayscale base image.

In both case, we can't see visually in image with secret that there is some hidden informations inside. For instance, image with hidden message seems to be normal in Figure 2.

5.1.1 Metrics

The results of the code metrics seem to confirm what we have been saying, as shown in Listing 2 and Listing 3. Note that metrics of grayscale image are very similar to metrics of only a color channel (with message).

- The mean squared error (MSE¹) measure the average squared distance between each pixel of the image. The difference between the distortion metric is that the MSE is normalized by the size of the image, but this two metrics are basically the same. We can note here that small values means that images are very similar, since average of squared distance between each pixel is small.

For RGB image, the mean squared error of the channel B of the image where the binary image is hidden is similar to the mean squared error of the grayscale image. However, the mean squared error of the entire image is $3 \times$ smaller than the mean squared error of the grayscale image.

This is due to the presence of 3 channels for RGB image versus 1 channel for grayscale image. The factor 3 is obtained because there is $3 \times$ more data in RGB image due to the 3 channels, giving more space to hide data and better camouflage.

That's why it's possible to hide more informations in RGB image thanks to the amount of data used to store each color channel.

- The Peak Signal Noise Ratio (PSNR²) is inversely proportional to the MSE. So big values means that images are very similar. However, this metric is quite different to the MSE in term of what is represented by this metric. The PSNR compares the maximum possible signal intensity to the noise introduced by image processing. The measure is in decibels (dB), and is easier to interpret in human point of view.

¹ $MSE = \sum_{i,j,k}^{M,N,C} \frac{[I_1(i,j,k) - I_2(i,j,k)]^2}{M \times N \times C}$ ² $PSNR = 10 \cdot \log_{10} \left(\frac{MAX_I^2}{MSE} \right)$



If we compare the PSNR obtained when hiding message in both RGB and grayscale images, we can see that the overall PSNR for the RGB image is bigger than for the grayscale image whereas if we focus on channel where the secret is hidden, the PSNR are similar. It's exactly the result expected result.

- The Structural Similarity (SSIM) is a perceptual metric used to quantify similarity between two images, but in human vision point of view. This metric is more complex to compute due to the covariance matrix computation (depending of the size of the image used, the implemented code take some time to answer). However, it gives an overview of similarity based on human vision, which is interesting in steganography because the goal is that human don't detect that there is a hidden message. SSIM metric return a value in range $[0, 1]$ and when the two images are identical, the return value is 1.

In case of RGB base image, the overall similarity is more close to 1 than for grayscale image (0.999 vs 0.998). However in both case, the values are very close to 1, meaning that in human point of view, the image with and without hidden secret are very similar. And it is exactly what we expect because else people can see when we try to hide some message in the image which goes against the principle of the steganography.

- The color difference¹ is a metric that compute euclidean distance between two points in color space. If we look at the result Figure 3 obtained, we can see that it is similar than the pixel-wise difference Figure 4 explained in Section 5.1.2. However, this metrics is more focus on how human perceive colors and can give us a better understanding of what's happen at color level. But in our case, the difference is too small to have some good results.

So in metric point of view, human can't see when a message is hidden in an image just by observing the image.

5.1.2 Pixel-wise difference

If we look at the pixel-wise difference² between the images in Figure 4 and Figure 5, we can easily see that the variation of the image with secret is only a very small noise. But without the original image, it is not possible to make the pixel-wise comparison of the two images. That's why human can't detect the presence of hidden message in an image without the help of some tools (unless the message is badly integrated to the image).

5.1.3 Histograms

However, if we take a look at the histograms of both RGB and grayscale image on Figure 4 and Figure 5, in both case the histogram of the image with secret hidden contains some small variations or oscillations that don't appear on histogram of non-modified image. In this manner, it's possible to detect that something has been added to the image thanks to the histogram oscillations.

However, if we look at the histogram of green channel in Figure 7, nothing appears on the histogram. It is because the base image has a huge size compared to the hidden image (6885×4534 pixels vs 1920×1080 pixels) So nothing is detected on histogram because modifications are too small compared to the amount of data.

¹ $\Delta = \sqrt{(L_x - L_y)^2 + (A_x - A_y)^2 + (B_x - B_y)^2}$ ² pixel-wise = $|I_1 - I_2|$



So we can easily see that the histograms allow to discover that there is some hidden message in the image. However, if nothing is detected, it doesn't mean that there is no hidden message. That's why other methods must be applied.

5.1.4 Robustness

5.1.4.1 RGB image

As shown in Figure 8, the message is lost when the image is cropped, compressed or noised. It's totally normal in both case:

- In case of cropped image, there is another length of the image used. So in function that get message, index will be chosen from a smaller list of index. The synchronization is therefore lost for generating the right sequence of index, as explained in Section 3. So the message obtained is some random message that doesn't matter with the expected message.
- In case of compressed image, as we have a RGB image, the JPEG compression will first convert image in YCbCr space domain. Then, color channels Cb and Cr are decreased 2 times. Due to this, all small variations introduced by the encoded message don't survive to this operation, resulting in a random message.
- In case of noised image, as the message is encoded into small variations of the image, adding small variations like a Gaussian noise destroy totally the message because the small variations of the message will be erased by the added small variations. However if we add some noise in only some part of the image, the message should be partially recovered because all small variations introduced by the message will not be fully lost.

5.1.4.2 Grayscale image

As shown in Figure 9, the message is lost when the image is cropped or noised for the same reasons as those explained in Section 5.1.4.1. However, the message is not lost when the image is compressed. This is due to the JPEG compression that don't change the color space of a grayscale image, but simply transform image in Discrete Cosine Transform domain to keep only low frequencies and erase high frequencies of the image since human vision is less sensible to high frequencies than to low frequencies. As the message introduced is essentially high frequencies (small variations = high frequencies), it means that if the compression remove high frequencies, the message could be lost. That's why to not loose the entire message, a very high quality of compression must be applied. For instance, with a quality of 95, the message is almost entirely erased, but with a quality of 98, it's possible to see a very noisy binary image. In this way, certain small variations generated by the secret message are lost during the compression process if the quality of the compression is not very very high. But not all, that's why the message can be partially recovered, depending on the compressive strength applied. In our case, as shown on Figure 9, the message is almost entirely recovered thanks to a high compression quality (compression quality of 100).

So the steganography is very sensitive to image modifications because the hidden message is encoded in small variations of the image, which are very sensitive to changes.



6 Conclusion

The steganography on images is a good way to secretly pass on informations. However, encoded messages are very sensitive to modifications of the image, like cropping, noise or compression.

The small oscillations on the histogram of the image can reveal that image perhaps contains some secret message. However, in case of big difference between size of image and size of message, theses oscillations can be totally hidden due to the large amount of data.

This is why we may wonder whether there are other ways to detect hidden messages and how effective these methods are.



Bibliography

- [1] S. Jajodia and N. F. Johnson, “ Exploring Steganography: Seeing the Unseen ,” *Computer*, vol. 31, no. 2, pp. 26–34, Feb. 1998, doi: 10.1109/MC.1998.10029.